Certification of Imitation Controllers Using Learned Lyapunov Functions

Master thesis by Josua Christoph Lindemann
Date: August 6, 2026, Date of submission: 16. August 2026

1. Review: Prof. Dr.-Ing. Rolf Findeisen
2. Review: Hendrik Alsmeier M.Sc.
Darmstadt

TECHNISCHE UNIVERSITÄT DARMSTADT

CCPS
Control and Cyber-Physical Systems

Electrical Engineering and Information Technology Department

Certification of Imitation Controllers Using Learned Lyapunov Functions

Master thesis by Josua Christoph Lindemann

1. Review: Prof. Dr.-Ing. Rolf Findeisen
2. Review: Hendrik Alsmeier M.Sc.

Date: August 6, 2026
Date of submission: 16. August 2026

Darmstadt

Technische Universität Darmstadt
Institut für Automatisierungstechnik und Mechatronik
Fachgebiet Control and Cyber-Physical Systems
Prof. Dr.-Ing. Rolf Findeisen

Contents

List of Variables and Symbols

Symbol	Description
System Dynamics & Control	
n_x	State space dimension
n_u	Control input space dimension
$\mathcal{X}$	Admissible state set
$\mathcal{U}$	Admissible input set
$\underline{x}$	Lower state bound
$\overline{x}$	Upper state bound
$\underline{u}$	Lower control input bound
$\overline{u}$	Upper control input bound
T_s	Sampling time
k	Discrete time step index
$x(t)$	Continuous-time state vector
x_k	Discrete-time state vector
$u(t)$	Continuous-time control input vector
u_k	Discrete-time control input vector
x^*	System equilibrium point
u^*	Input reference at x^*
x_{k+1}	Discrete-time successor state
x^+	One-step successor state
$\pi(x)$	State-feedback policy
$f_c(x, u)$	Continuous-time system dynamics
$f(x_k, u_k)$	Discrete-time system dynamics
Optimal Control & Model Predictive Control (MPC)	
N	Prediction horizon
$\ell(x, u)$	Stage cost function
$V_f(x)$	Terminal cost function
$\mathcal{X}_f$	Terminal constraint set
Q	State weighting matrix
R	Input weighting matrix
u_0^*	Optimal control input
$V_N^*(x)$	Optimal value function
A	Linearized system state matrix
B	Linearized system input matrix
K	LQR feedback gain matrix

Symbol	Description
P	DARE solution matrix
α_N	Relaxed Lyapunov decrease factor
$\mathcal{X}_N$	Feasible initial state set
Neural Network	
L	Total layer count
l	Layer index
$a^{(l)}$	Layer activation vector
$z^{(l)}$	Layer pre-activation vector
$W^{(l)}$	Layer weight matrix
$b^{(l)}$	Layer bias vector
$\sigma^{(l)}(\cdot)$	Non-linear activation function
θ	Trainable network parameters
θ^*	Optimal network parameters
$\mathcal{D}$	Training dataset
M	Training data size
$\mathcal{B}$	Mini-batch
$\mathcal{L}_{\text{MSE}}$	Mean squared error loss
$\mathcal{L}_{\text{reg}}$	Regularization loss
$\mathcal{L}_{\text{aux}}$	Auxiliary loss
ω_{reg}	Regularization weight
ω_{aux}	Auxiliary weight
$\delta^{(l)}$	Loss sensitivity per layer
$\delta^{(L)}$	Loss sensitivity at last layer
Imitation Learning	
$\mathcal{X}_\pi$	Imitation learning state space
$\underline{x}_\pi$	Admissible state lower bound
$\overline{x}_\pi$	Admissible state upper bound
$\mathcal{D}_\pi$	Imitation learning dataset
$N_\mathcal{D}$	Dataset sample count
$\mathcal{B}_\pi$	Training mini-batch
M_π	Mini-batch size
$\pi_0(x)$	Expert MPC policy
$\pi_\theta(x)$	Learned imitation policy
$h_\theta(x)$	Unconstrained neural network output
$\text{clamp}(\cdot)$	Control input clamping function
S_x	State scaling matrix
S_u	Input scaling matrix
x_{scale}	State scaling vector
u_{scale}	Input scaling vector
$d(x)$	Scaled equilibrium distance
$w_x(x)$	Behavioral cloning loss weight
$w_{\min}$	Baseline loss weight
η	Decay rate for state-dependent weight

Symbol	Description
$\mathcal{L}_{\text{BC}}$	Behavioral cloning loss
$\mathcal{L}_f$	Dynamics-aware constraint loss
ω_{BC}	Behavioral cloning loss weight
ω_f	Dynamics-aware constraint loss weight
$\mathcal{L}_\pi$	Total imitation policy loss
Neural Lyapunov Learning & Falsification	
$\pi_V(x)$	Co-trained imitation policy
$V_\theta(x)$	Neural Lyapunov candidate
ε	Regularization constant
R_θ	Learnable transformation matrix
ρ	Target Lyapunov sublevel value
$\mathcal{V}_\rho$	Certified sublevel set
$\mathcal{X}_V$	Lyapunov training domain
ρ_{min}	Minimum sublevel target bound
e_V	Lyapunov decrease violation
κ	Target decrease rate
ε_{rel}	Numerical stabilization constant
$\mathcal{L}_{\text{dec}}$	Relative Lyapunov decrease loss
S_{range}	Domain range scaling matrix
$\mathcal{L}_{\text{inv}}$	Relative state-invariance loss
$\mathcal{L}_{\text{cond}}$	Combined condition violation loss
w_ρ	Soft sublevel gating weight
s	Gate sharpness parameter
$\mathcal{L}_{\rho\text{-gated}}$	Soft-gated condition loss
$\mathcal{B}_\partial$	Boundary candidate sample set
$\mathcal{B}_{\text{ROA}}$	ROA shell sample set
ξ	Inner boundary shell factor
$\mathcal{L}_{\text{ROA}}$	Region of Attraction expansion loss
$e_{V,\text{lower}}$	Signed Lyapunov decrease lower bound
$\mathcal{M}_{\text{L}}$	Region LIRPA condition violation
w_{L}	LIRPA region weight
α_{L}	LIRPA weight decay rate
N_{L}	LIRPA region count
$\mathcal{L}_{\text{L}}$	LIRPA loss
$\mathcal{B}_V$	Lyapunov scaling Sobol sample batch
ℓ_V	Mean Lyapunov log-value
$\ell_{V,0}$	Initial mean Lyapunov log-value
$\mathcal{L}_V$	Lyapunov magnitude scaling loss
$\mathcal{B}_{\text{pol}}$	Policy scaling Sobol sample batch
$\mathcal{L}_{\text{pol}}$	Policy scaling loss
$\ell_{R,0}$	Initial matrix R Frobenius norm
$\mathcal{L}_R$	Matrix R regularization loss
$\mathcal{L}_{\text{total}}$	Total combined Lyapunov loss
ω_{cond}	Soft-gated condition loss weight

Symbol	Description
ω_{ROA}	Region of Attraction expansion loss weight
ω_{L}	LIRPA loss weight
ω_V	Lyapunov magnitude scaling loss weight
ω_{pol}	Policy scaling loss weight
ω_R	Matrix R regularization loss weight
$\mathcal{C}$	Counterexample pool
$\mathcal{S}$	Explorative pool
$\mathcal{B}_{\mathcal{S}}$	Explorative pool candidate sample set
$N_{\mathcal{S}}$	Explorative pool target size
x_{adv}	Counterexample state
J_{PGD}	PGD objective
τ	Counterexample age
$\tau_{\max}$	Maximum lifespan
m	Sublevel margin
γ	Overestimation factor
$\tilde{\rho}$	Estimated ROA boundary minimum
β	EMA decay rate
N_{outer}	Outer epoch count
K_{steps}	Inner mini-batch optimization steps per epoch
N_{cex}	Counterexample mining step interval
Formal Verification & Certification	
$\mathcal{B}_C$	Formal verification domain
$\Phi(x; \rho)$	Verification specification formula
Φ_{sub}	Sublevel set exclusion predicate
Φ_{pos}	Positive definiteness predicate
Φ_{inv}	Forward invariance predicate
Φ_{dec}	Strict Lyapunov decrease predicate
$\mathcal{B}_{\text{hole}}$	Equilibrium exclusion region
ρ^*	Maximal verified sublevel value

1 Abstract

2 Introduction

3 Optimal Control and Lyapunov Stability Theory

3.1 Discrete-Time Dynamical Systems and Stability

Implementing optimal control schemes on digital hardware requires a discrete-time representation. Consequently, the continuous-time dynamics

$$\dot{x}(t) = f_c(x(t), u(t)) \tag{3.1}$$

must be discretized, where $x(t) \in \mathbb{R}^{n_x}$ and $u(t) \in \mathbb{R}^{n_u}$ denote the continuous-time state and input vectors, respectively, with state space dimension $n_x \in \mathbb{N}$ and input dimension $n_u \in \mathbb{N}$. In this work, the corresponding discrete-time system

$$x_{k+1} = f(x_k, u_k) \tag{3.2}$$

is obtained by approximating the continuous dynamics using a numerical integration scheme with a constant sampling time $T_s > 0$. Here, $k \in \mathbb{N}_0$ denotes the discrete time step index, such that $x_k := x(kT_s) \in \mathbb{R}^{n_x}$ and $u_k := u(kT_s) \in \mathbb{R}^{n_u}$ represent the state and control input at time step k. The discrete-time function $f : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \to \mathbb{R}^{n_x}$ is evaluated using a numerical integration scheme, such as the forward Euler or a fourth-order Runge-Kutta (RK4) method [1].

To account for physical and operational limits, the system states and control inputs are restricted to compact constraint sets $\mathcal{X} := [\underline{x}, \overline{x}] \subseteq \mathbb{R}^{n_x}$ and $\mathcal{U} := [\underline{u}, \overline{u}] \subseteq \mathbb{R}^{n_u}$. Furthermore, without loss of generality, any equilibrium (x^*, u^*) can be shifted to the origin via deviation variables $\tilde{x} := x - x^*$ and $\tilde{u} := u - u^*$. To keep notation concise, the tildes are omitted and x and u are used directly for the shifted variables, assuming $(x^*, u^*) = (\mathbf{0}, \mathbf{0})$ throughout this chapter.

3.1.1 Lyapunov Stability

Consider the discrete-time closed-loop system under a state-feedback control policy $\pi : \mathcal{X} \to \mathcal{U}$ with the origin $x = \mathbf{0}$ as an equilibrium point, satisfying $\pi(\mathbf{0}) = \mathbf{0}$ and $f(\mathbf{0}, \mathbf{0}) = \mathbf{0}$. Assuming the state constraint set $\mathcal{X}$ is positively invariant under π, the origin is locally asymptotically stable in $\mathcal{X}$ if there exists a positive definite function $V : \mathcal{X} \to \mathbb{R}_{\geq 0}$ satisfying [2, Theorem 2.13]:

$$\begin{aligned} V(\mathbf{0}) &= 0, \\ V(x) &> 0 \qquad \forall x \in \mathcal{X} \setminus \{\mathbf{0}\}, \\ V(f(x, \pi(x))) - V(x) &< 0 \qquad \forall x \in \mathcal{X} \setminus \{\mathbf{0}\}. \end{aligned} \tag{3.3}$$

The strict decrease condition guarantees trajectory convergence to the origin for all initial states x_0 within a certified domain.

look up if all assumtions are named

3.2 Model Predictive Control and Stability

This section summarizes the model predictive control (MPC) framework and the conditions required to guarantee closed-loop stability in finite-horizon optimal control setups.

3.2.1 Problem Formulation

For the nominal MPC setup considered in this work, the following finite-horizon optimal control problem is solved at each time step k given the current system state x_k [2, Sec. 2.2], [3, Sec. 3.3]:

$$\begin{aligned}
\min_{\{x_i\}_{i=0}^{N},\{u_i\}_{i=0}^{N-1}} \quad & \sum_{i=0}^{N-1} \ell(x_i,u_i) + V_f(x_N) \\
\text{s.t.} \quad & x_{i+1} = f(x_i,u_i), && i = 0,\dots,N-1, \\
& x_i \in \mathcal{X}, && i = 0,\dots,N-1, \\
& u_i \in \mathcal{U}, && i = 0,\dots,N-1, \\
& x_N \in \mathcal{X}_f, \\
& x_0 = x_k.
\end{aligned} \tag{3.4}$$

Here, $N \in \mathbb{N}$ denotes the prediction horizon, $\ell : \mathcal{X} \times \mathcal{U} \to \mathbb{R}_{\geq 0}$ is the stage cost function, and $V_f : \mathcal{X}_f \to \mathbb{R}_{\geq 0}$ represents the terminal cost function defined over the terminal constraint set $\mathcal{X}_f \subseteq \mathcal{X}$. To distinguish predicted states and inputs from the actual closed-loop state x_k and control input u_k, the sequences $\{x_i\}_{i=0}^{N}$ and $\{u_i\}_{i=0}^{N-1}$ serve as optimization variables local to time step k, constrained by the sets $\mathcal{X}$ and $\mathcal{U}$ [3, Sec. 3.2].

For this work, the stage cost is restricted to a quadratic cost function with positive definite weighting matrices $Q \succ 0$ and $R \succ 0$, which is defined as follows [2, Sec. 1.3]:

$$\ell(x_i,u_i) = x_i^\top Q x_i + u_i^\top R u_i, \tag{3.5}$$

Following the receding horizon principle, only the first control input u_0^* from the optimal sequence is applied to the system, and the optimization is repeated at the next sampling instance [2, Sec. 2.2], [3, Sec. 3.1].

3.2.2 Terminal Ingredients for Stability

To guarantee closed-loop stability of a nominal MPC, the optimal control problem (3.4) incorporates specific terminal ingredients: the terminal cost function $V_f(x_N)$ and the terminal set $\mathcal{X}_f$ [2, Assumption 2.14], [3, Assumption 5.9].

The terminal set $\mathcal{X}_f$ is required to be positively invariant under a local control law $u = \pi(x)$, meaning that any trajectory originating in $\mathcal{X}_f$ remains inside the set for all future time steps while respecting input constraints [2, Def. 2.9]:

$$f(x,\pi(x)) \in \mathcal{X}_f \quad \text{and} \quad \pi_f(x) \in \mathcal{U}, \qquad \forall x \in \mathcal{X}_f. \tag{3.6}$$

add contraction: $V_f(f(x,\pi_f(x))) - V_f(x) \leq -ell(x,\pi_f(x))$

The predictive mechanism is illustrated in Fig. 3.1. While the intermediate states $x_0, \ldots, x_{N-1}$ are merely constrained to the safe state space $\mathcal{X}$, the terminal constraint enforces $x_N \in \mathcal{X}_f$. Let $V_N^*(x)$ denote the optimal value function of (3.4) for a state x. Combined with the terminal cost $V_f(x_N)$ acting as a local Lyapunov function in $\mathcal{X}_f$, these ingredients ensure that the optimal value function $V_N^*(x)$ serves as a Lyapunov function for the closed-loop system, guaranteeing asymptotic stability [2, Theorem 2.19].

For nonlinear systems, the terminal set $\mathcal{X}_f$ is typically computed using linearized system dynamics around the equilibrium point $(x, u) = (\mathbf{0}, \mathbf{0})$, yielding the linear discrete-time model:

$$x_{k+1} = Ax_k + Bu_k. \tag{3.7}$$

A widely used choice for these terminal ingredients is then derived from the Linear-Quadratic Regulator (LQR) [2, Eq. (1.18)], [3, Eq. (5.24)]. Here, the local control law $\pi(x_k)$ is chosen as a linear state-feedback policy $\pi(x_k) = -Kx_k$, where the optimal LQR gain K is given by

$$K = \left(R + B^\top PB\right)^{-1} B^\top PA. \tag{3.8}$$

Under the standard stabilizability and detectability assumptions, the matrix $P \succ 0$ is obtained as the unique stabilizing solution to the discrete-time Algebraic Riccati Equation (DARE) [2, Eq. (1.18)], [3, Eq. (5.23)]:

$$P = A^\top PA - A^\top PB\left(R + B^\top PB\right)^{-1} B^\top PA + Q. \tag{3.9}$$

This matrix parameterizes the quadratic terminal cost function $V_f(x_N) = x_N^\top P x_N$. Accordingly, the terminal set $\mathcal{X}_f$ is defined in this specific case as an ellipsoidal sub-level set of this cost function [2, Sec. 2.5.5]:

$$\mathcal{X}_f := \left\{x \in \mathbb{R}^{n_x} \,\middle|\, x^\top Px \leq \alpha\right\}. \tag{3.10}$$

Choosing the scaling parameter $\alpha > 0$ sufficiently small guarantees that $\mathcal{X}_f$ satisfies both the state and input constraints $\mathcal{X}$ and $\mathcal{U}$ while ensuring that the quadratic Lyapunov decrease dominates higher-order linearization errors of the nonlinear dynamics.

3.2.3 Recursive Feasibility and Closed-Loop Stability

more detail on recursive feasibility, smoother transition to closed loop stability.

Under the MPC policy $\pi_0(x_k) := u_0^*(x_k)$, the optimal value function $V_N^*(x_k)$ of (3.4) serves as a Lyapunov candidate over the feasible set $\mathcal{X}_N \subseteq \mathcal{X}$ [2, Sec. 2.4.2]. Using the terminal ingredients from Section 3.2.2, the recursive feasibility at time step $k+1$ is determined by using the remaining inputs of the optimal sequence computed at time step k and appending the terminal control action $\pi(x_N^*)$ [2, Sec. 2.4.2].

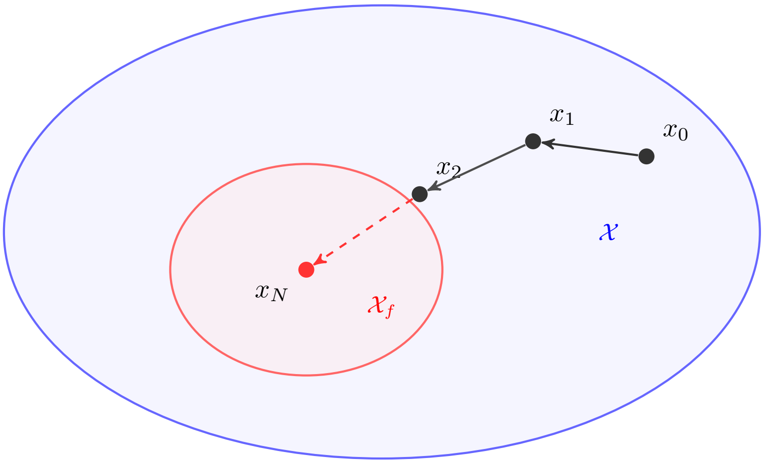

Figure 3.1: Comparison of the terminal set $\mathcal{X}_f$ (red) with the entire state space $\mathcal{X}$ (blue).

Consequently, V_N^* satisfies a decrease condition along closed-loop trajectories $x_{k+1} = f(x_k, \pi_0(x_k))$. Relaxing the standard stability conditions, this can be expressed with the relaxed Lyapunov decrease formulation [3, Theorem 4.11]:

$$V_N^*(x_{k+1}) \leq V_N^*(x_k) - \alpha_N \ell(x_k, \pi_0(x_k)), \qquad \forall x_k \in \mathcal{X}_N, \tag{3.11}$$

where $\alpha_N \in (0, 1]$ denotes the relaxed decrease factor [3, Theorem 4.11]. Setting $\alpha_N = 1$ yields the standard nominal Lyapunov decrease

$$V_N^*(x_{k+1}) - V_N^*(x_k) \leq -\ell(x_k, \pi_0(x_k)), \tag{3.12}$$

guaranteeing asymptotic stability of the origin **0** for all initial states $x_0 \in \mathcal{X}_N$ [2, Assumption 2.14].

4 Foundations of Neural Control Policies

4.1 Artificial Neural Networks

In imitation learning, artificial neural networks (ANNs) approximate complex control laws offline to bypass the computational overhead of online model predictive control (MPC) evaluations.

4.1.1 Multi-Layer Perceptrons

Multi-layer perceptrons (MLPs) are fully connected feed-forward networks that map an input through $L-1$ hidden layers to an output layer [4, Sec. 1.4], [5, Sec. 3.1]. For a state input x, forward propagation to the network prediction $\hat{y}$ is defined recursively for layers $l = 1, \ldots, L$:

$$a^{(l)} = \sigma^{(l)}\left(z^{(l)}\right) \quad \text{with} \quad z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad \forall l = 1, \ldots, L, \tag{4.1}$$

where $a^{(0)} := x$ is the input, $z^{(l)}$ is the pre-activation vector of layer l, and $a^{(L)} := \pi_\theta(x) = \hat{y}$ is the network prediction. The matrix $W^{(l)}$ and vector $b^{(l)}$ denote the weights and biases of layer l, respectively, while $\sigma^{(l)}(\cdot)$ represents a non-linear activation function applied element-wise to the pre-activation vector $z^{(l)}$ [5, Sec. 3.2]. Without these non-linearities, the layers would simply be reduced to a single linear transformation, drastically limiting the model's ability to capture complexity [4, Sec. 1.5]. Common activation functions in neural control include the hyperbolic tangent (4.2a) and the rectified linear unit (ReLU) (4.2b), defined respectively as [5, Sec. 3.5].

$$\tanh z = \frac{\mathrm{e}^{2z} - 1}{\mathrm{e}^{2z} + 1} \tag{4.2a}$$

$$[z]_+ = \max(0, z) \tag{4.2b}$$

Their qualitative profiles are illustrated in Fig. 4.1.

Based on the universal approximation theorem, an MLP with at least one sufficiently sized hidden layer and non-linear activations can approximate any continuous multidimensional function on compact sets to arbitrary accuracy [5, Sec. 3.1]. The trainable parameters are represented by $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L}$. Figure 4.2 illustrates a representative architecture.

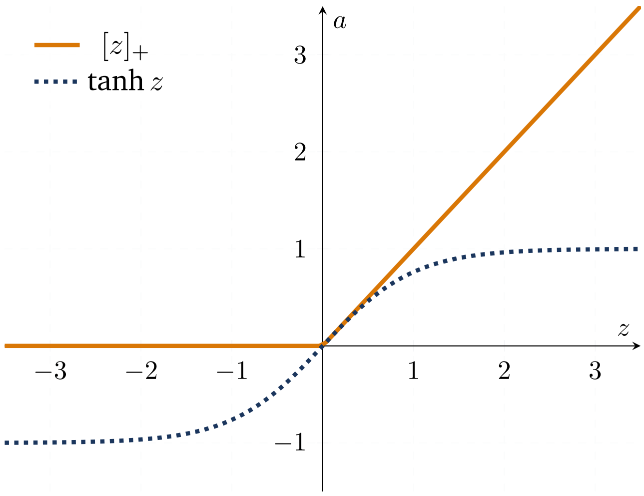

Figure 4.1: Standard activation functions for neural control policies.

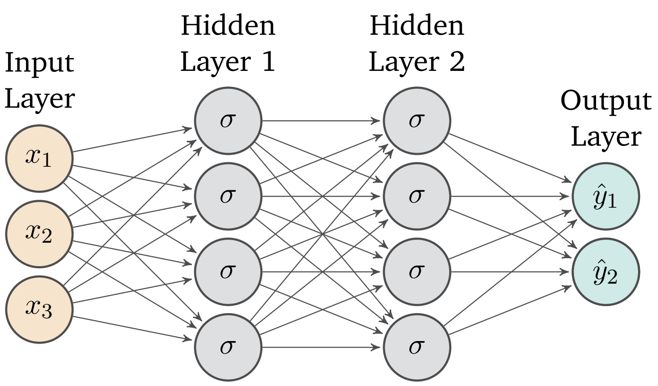

Figure 4.2: Architecture of an exemplary Multi-Layer Perceptron with three inputs, two outputs and two hidden layers with four neurons each. Illustration adapted from [5, Fig. 3.1]

4.1.2 Optimization and Training

To approximate an expert control law with a neural network $\pi_\theta(x)$, the parameters θ are updated via gradient-based optimization [5, Sec. 2.5]. The objective is to find parameters $\theta^\star$ that minimize the empirical training objective:

$$\theta^\star \in \arg\min_\theta \mathcal{L}(\mathcal{D};\theta). \tag{4.3}$$

Dataset and Primary Loss Function. The training process relies on an offline dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^M$ containing M input-target pairs sampled across the domain. For continuous regression tasks, a common loss term is the mean squared error (MSE) evaluated over the training dataset:

$$\mathcal{L}_{\text{MSE}}(\mathcal{D};\theta) = \frac{1}{M} \sum_{(x,y)\in\mathcal{D}} \|\pi_\theta(x) - y\|_2^2 \,. \tag{4.4}$$

Here, x denotes a system state and y denotes the corresponding target value.

Loss Augmentation and Regularization. In supervised learning, relying solely on empirical error minimization over a finite dataset can lead to overfitting and fails to enforce underlying structural or domain-specific constraints. To improve generalization and incorporate prior knowledge directly into the optimization process, the objective is frequently augmented with weight regularization and auxiliary loss terms [5, Sec. 3.7]:

$$\mathcal{L}(\mathcal{D};\theta) = \mathcal{L}_{\text{MSE}}(\mathcal{D};\theta) + \omega_{\text{reg}}\mathcal{L}_{\text{reg}}(\theta) + \omega_{\text{aux}}\mathcal{L}_{\text{aux}}(\mathcal{D};\theta). \tag{4.5}$$

where $\omega_{\text{reg}}, \omega_{\text{aux}} \geq 0$ serve as weighting hyperparameters balancing prediction error, parameter regularization, and constraint satisfaction.

The regularization term $\mathcal{L}_{\text{reg}}$ constrains the network's parameter space, typically realized as an L_1-norm ($\|\theta\|_1$) or L_2-norm ($\|\theta\|_2^2$) penalty. While L_1-regularization promotes sparse parameter representations, L_2-regularization penalizes large parameter magnitudes and can improve numerical stability and generalization. In practice, these penalties are applied exclusively to the network's weight matrices $W \subset \theta$, leaving bias vectors b unpenalized. [5, Sec. 3.7.2].

In addition to regularization penalties, the auxiliary loss $\mathcal{L}_{\text{aux}}$ embeds domain knowledge into the optimization process. Popularized by Physics-Informed Neural Networks (PINNs), it directly penalizes violations of explicit rules, such as differential equations or Lyapunov conditions [5, Sec. 5], [6]. Enforcing these properties during training helps the network learn the desired solutions.

Gradient Computation via Backpropagation. For updating the network parameters θ using gradient-based optimization, the partial derivatives of the loss $\mathcal{L}(\mathcal{D},\theta)$ with respect to all weight matrices $W^{(l)}$ and bias vectors $b^{(l)}$ must be evaluated. Backpropagation efficiently computes chain-rule derivatives of loss components that depend on the parameters through the network output by combining a forward pass (4.1) with a backward pass [5, Sec. 3.4].

During the backward pass, the algorithm computes the sensitivity of the loss with respect to the pre-activations, defined as the error signal $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}}$. At the output layer L, this signal is initialized as:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \odot \frac{\partial a^{(L)}}{\partial z^{(L)}}, \tag{4.6}$$

where $\odot$ represents the element-wise (Hadamard) product. Propagating the error backward for $l = L-1, \ldots, 1$ yields the recursive relation:

$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot \frac{\partial a^{(l)}}{\partial z^{(l)}} \tag{4.7}$$

Combining these error signals with stored activations $a^{(l-1)}$ yields the network-dependent gradients of the output-dependent loss terms. Adding the explicit derivative of the parameter regularization term $\mathcal{L}_{\text{reg}}$ completes the total gradient evaluation:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)} \right)^T + \omega_{\text{reg}} \frac{\partial \mathcal{L}_{\text{reg}}}{\partial W^{(l)}}, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}. \tag{4.8}$$

By reusing intermediate activations and error signals, backpropagation avoids redundant derivative calculations and computes parameter gradients with computational cost of the same order as a forward evaluation of the network [5, Sec. 3.4], [4, Sec. 2.3].

Optimization via Mini-Batch Gradient Descent. While full-batch gradient descent evaluates the loss gradient over the entire dataset $\mathcal{D}$, this can become computationally expensive and impractical for large dataset sizes M. Conversely, single-sample stochastic gradient descent (SGD) results in high variance of the parameter updates, leading to erratic optimization trajectories. Mini-batch gradient descent balances this trade-off by dividing the dataset into smaller subsets $\mathcal{B} \subset \mathcal{D}$ of size $B = |\mathcal{B}|$. For a mini-batch, the loss $\mathcal{L}(\mathcal{B}; \theta)$ is evaluated analogously to Eq. (4.5), with $\mathcal{D}$ replaced by $\mathcal{B}$ in all data-dependent loss terms. The mini-batch gradient $\mathbf{g}$ typically has lower variance than the gradient obtained from a single sample, while mini-batch processing enables efficient parallelization on modern GPU hardware. During training, the parameters θ are iteratively updated along the negative gradient direction scaled by a learning rate $\alpha > 0$. This update loop is executed for a total of $E \in \mathbb{N}$ epochs (full passes through the dataset), as detailed in Algorithm 1 [4, Sec. 2.6.1], [5, Sec. 2.6].

Adaptive Moment Estimation (Adam). Although mini-batch gradient descent improves stability compared to SGD, its performance depends on a static learning rate α. In loss landscapes with ill-conditioned curvature or noisy mini-batch gradients, a fixed learning rate can cause parameter updates to oscillate in steep directions while making slow progress along plateaus [4, Sec. 4.5.4].

To resolve these limitations, the Adaptive Moment Estimation (Adam) optimizer [7] computes individual, adaptive learning rates for each parameter by tracking exponential moving averages of both past gradients (1^{st} moment m_t) and past squared gradients (2^{nd} moment v_t) [4, Sec. 4.5.5.2]:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \mathbf{g}_t, \tag{4.9}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \tag{4.10}$$

Algorithm 1: Mini-Batch Stochastic Gradient Descent

Input: Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{M}$, learning rate α, batch size B, max epochs E
Output: Optimized parameters θ

Initialize network parameters $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L}$;
for *epoch* $\leftarrow 1$ **to** E **do**
    Shuffle dataset $\mathcal{D}$;
    Partition $\mathcal{D}$ into mini-batches $\mathcal{B}_1, \ldots, \mathcal{B}_K$ of at most B samples;
    foreach $\mathcal{B} \in \{\mathcal{B}_1, \ldots, \mathcal{B}_K\}$ **do**
        Evaluate output predictions $\hat{y} = \pi_\theta(x) \quad \forall (x,y) \in \mathcal{B}$;
        Compute mini-batch gradient: $\mathbf{g} \leftarrow \nabla_\theta \mathcal{L}(\mathcal{B}; \theta)$;
        Update parameters: $\theta \leftarrow \theta - \alpha \mathbf{g}$;
    end
end
return θ;

where $\beta_1, \beta_2 \in [0, 1)$ dictate the exponential decay rates, and $\mathbf{g}_t^2$ denotes the element-wise square of the current mini-batch gradient $\mathbf{g}_t = \nabla_\theta \mathcal{L}(\mathcal{B}_t; \theta_{t-1})$.

Because m_0 and v_0 are initialized as zero vectors, both moment estimates are initially biased toward zero. To counteract this initialization bias, dynamic correction factors are applied at each timestep t:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \tag{4.11}$$

The network parameters are then updated element-wise according to:

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \tag{4.12}$$

where $\epsilon > 0$ is a small constant ensuring numerical stability. Following [7], standard default hyperparameters are chosen as $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$, providing robust convergence across various deep learning architectures. The complete iterative training loop using Adam is summarized in Algorithm 2.

4.2 Adversarial Training and Falsification

Algorithm 2: Adaptive Moment Estimation (Adam)

Input: Dataset $\mathcal{D}$, initial parameters θ_0, step size $\alpha = 0.001$, decay rates $\beta_1 = 0.9, \beta_2 = 0.999$, stability constant $\epsilon = 10^{-8}$

Output: Optimized parameters θ

Initialize 1st moment vector $m_0 \leftarrow 0$;

Initialize 2nd moment vector $v_0 \leftarrow 0$;

Initialize timestep $t \leftarrow 0$;

while θ *not converged* **do**

  $t \leftarrow t + 1$;

  Sample mini-batch $\mathcal{B}_t \subset \mathcal{D}$;

  Compute mini-batch gradient: $\mathbf{g}_t \leftarrow \nabla_\theta \mathcal{L}(\mathcal{B}_t; \theta_{t-1})$;

  `// Update biased moment estimates`

  $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1)\mathbf{g}_t$;

  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)\mathbf{g}_t^2$;

  `// Compute bias-corrected estimates`

  $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$;

  $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$;

  `// Update parameters`

  $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$;

end

return θ_t;

5 Formal Verification of Neural Control Policies

5.1 Mixed-Integer Linear Programming

5.2 Branch and Bound

5.3 $\alpha\beta$-CROWN

6 Learning Lyapunov-Stable Imitation Controllers

This chapter presents the methodology for learning and certifying a neural approximation of the MPC controller. The MPC policy introduced in Section 3.2.1 serves as an expert, generating state-input pairs for supervised policy training. As described in Section 6.1, this allows the neural policy to reproduce the expert control actions over the considered state domain. Subsequently, a neural Lyapunov candidate is learned to enable the stability certification of the controller. In the co-training setting, the policy parameters may additionally be refined, while the Lyapunov architecture and training objective enforce the stability conditions detailed in Section 6.2. Finally, to formally prove that the learned controller satisfies these guarantees across the continuous state space, the networks are subjected to formal verification using α,β-CROWN, as presented in Section 6.3.

6.1 Neural Policy Approximation and Imitation Learning

Data Generation uses the MPC expert policy π_0 defined in Section 3.2.3. Closed-loop trajectories $x_{k+1} = f(x_k, \pi_0(x_k))$ are initialized from states sampled across the training set $\mathcal{X}_\pi : \{x \in \mathcal{X} \mid \underline{x}_\pi \leq x \leq \overline{x}_\pi\}$. Storing the visited states and their corresponding optimal control actions yields the dataset

$$\mathcal{D}_\pi = \left\{ (x^{(i)}, u^{(i)}) \right\}_{i=1}^{N_\mathcal{D}}, \tag{6.1}$$

where $x^{(i)} \in \mathcal{X}_\pi$ and $u^{(i)} = \pi_0(x^{(i)})$.

Imitation Policy follows the structure proposed in [6], as it enforces true control inputs at the equilibrium $\pi_\theta(x^*) = u^*$ and satisfies the control input constraints $\underline{u} \leq \pi_\theta(x) \leq \overline{u}$ by construction. The neural policy π_θ is visualized in Fig. 6.1 and formulated as

$$\pi_\theta(x) = \text{clamp}(h_\theta(x) - h_\theta(x^*) + u^*, \underline{u}, \overline{u}). \tag{6.2}$$

where $h_\theta : \mathbb{R}^{n_x} \to \mathbb{R}^{n_u}$ is an unconstrained neural network, $u^* \in \mathcal{U}$ represents the desired control action at the equilibrium x^*, and $\text{clamp}(\cdot, \bar{\cdot}, \underline{\cdot})$ acts element-wise to enforce the input constraints.

Imitation Objective consists of a state-weighted behavioral cloning loss and a dynamics-aware penalty. To account for different physical units and numerical scales across the state and control vectors, the loss terms are normalized using strictly positive diagonal scaling matrices

$$S_x = \text{diag}(x_{\text{scale}}) \in \mathbb{R}^{n_x \times n_x} \quad \text{and} \quad S_u = \text{diag}(u_{\text{scale}}) \in \mathbb{R}^{n_u \times n_u}. \tag{6.3}$$

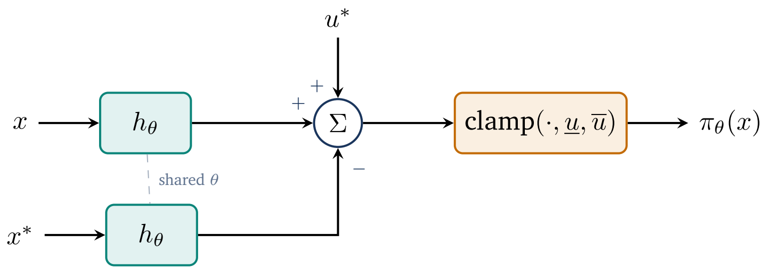

Figure 6.1: The imitation learning policy.

Using S_x, the normalized distance of a state x from the equilibrium x^* is quantified with the L_1-norm as

$$d(x) = \frac{1}{n_x} \left\| S_x^{-1}(x - x^*) \right\|_1 . \tag{6.4}$$

To assign appropriate priority during behavioral cloning, a state-dependent weight is defined as

$$w_x(x) = w_{\text{min}} + (1 - w_{\text{min}}) \left(\frac{1}{1 + d(x)} \right)^{\eta}, \tag{6.5}$$

where $w_{\text{min}} \in (0, 1)$ is the baseline weight for states far from the equilibrium and $\eta \geq 0$ controls the decay rate.

State-Weighted Behavioral Cloning Loss penalizes the squared normalized Euclidean distance between the neural network policy and the expert targets. For a mini-batch $\mathcal{B}_\pi \subset \mathcal{D}_\pi$ of size $M_\pi = |\mathcal{B}_\pi|$, the behavioral cloning loss is defined as

$$\mathcal{L}_{\text{BC}} = \frac{1}{M_\pi n_u} \sum_{(x,u) \in \mathcal{B}_\pi} w_x(x) \left\| S_u^{-1} \big(\pi_\theta(x) - u \big) \right\|_2^2 . \tag{6.6}$$

Here, the state-dependent weight $w_x(x)$ prioritizes approximation accuracy near the equilibrium.

Dynamics-Aware Constraint Loss applies a one-step penalty for successor states that leave the admissible state set $\mathcal{X}_\pi$. For each state $x \in \mathcal{B}_\pi$, the next state is evaluated under the neural policy as $x^+ = f(x, \pi_\theta(x))$. Using the element-wise ReLU function defined in (4.2b), the loss is defined as

$$\mathcal{L}_f = \frac{1}{M_\pi n_x} \sum_{x \in \mathcal{B}_\pi} \left(\left\| \left[x^+ - \overline{x}_\pi \right]_+ \right\|_1 + \left\| \left[\underline{x}_\pi - x^+ \right]_+ \right\|_1 \right) . \tag{6.7}$$

Here, a one-step evaluation outside the admissible set results in a penalty for the current state, forcing the learned policy to stay inside the admissible set.

Total Imitation Loss is the combination of the state-weighted behavioral cloning and dynamics-aware constraint losses into a single scalar objective. Scaling each term by its respective weight, ω_{BC} and ω_f, yields the overall policy loss:

$$\mathcal{L}_\pi = \omega_{\text{BC}} \mathcal{L}_{\text{BC}} + \omega_f \mathcal{L}_f . \tag{6.8}$$

6.2 Lyapunov Learning for Stability Certification

To establish formal stability guarantees for learned controllers, a neural Lyapunov candidate is trained alongside the policy. A falsification-guided framework is used to find stability violations and therefore make the Lyapunov candidate a valid certificate for stability.

lyapunov and imitation policy cotraining

6.2.1 Lyapunov Network Architecture

Following [6], the Lyapunov candidate $V_\theta(x)$ is parameterized to be positive definite by construction:

$$V_\theta(x) = |g_\theta(x) - g_\theta(x^*)| + \left\| (\varepsilon I + R_\theta^\top R_\theta)(x - x^*) \right\|_1, \tag{6.9}$$

where $g_\theta : \mathbb{R}^n \to \mathbb{R}$ is a neural network, $x \in \mathbb{R}^n$ is the current state, $x^* \in \mathbb{R}^n$ is the equilibrium, $\varepsilon > 0$ is a small regularization constant, and $R_\theta \in \mathbb{R}^{n\times n}$ is a learnable matrix. While the first term is only positive semi-definite and may vanish away from the equilibrium, the second term guarantees strict positive definiteness $\forall x \in \mathcal{X} \setminus \{x^*\}$, as the matrix $(\varepsilon I + R_\theta^\top R_\theta)$ is strictly positive definite by construction for any $\varepsilon > 0$. The complete architectural layout of $V_\theta(x)$ is illustrated in Fig. 6.2.

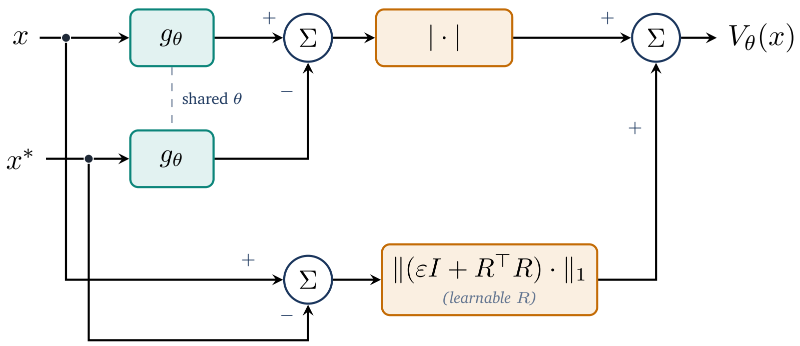

Figure 6.2: The neural Lyapunov architecture.

6.2.2 Loss Formulation

add more explanations

To ensure that the Lyapunov candidate described in (6.9) can be certified for stability, the loss function must comprise multiple components. It utilizes the previously introduced ReLU function $[\cdot]_+$ for one-sided penalties, alongside a weighted mean to aggregate the sample losses. Central to the following loss terms is the estimation of the region of attraction (ROA). Throughout the training process, a target level ρ is considered, leading to the corresponding ρ-sublevel set within the training domain $\mathcal{X}_V := \{x \in \mathcal{X}_\pi \mid \underline{x}_V \le x \le \overline{x}_V\}$, defined as

$$\mathcal{V}_\rho := \{x \in \mathcal{X}_V \mid V_\theta(x) \le \rho\}. \tag{6.10}$$

To prevent this certified region from trivially collapsing towards the origin during optimization, and to ensure numerical stability during normalizations, ρ is strictly constrained by a lower bound $\rho_{\text{min}} > 0$. Furthermore, the successor states $x_V^+ = f(x, \pi_V(x))$ needed during loss computation are evaluated on-the-fly using the system dynamics and the policy. The latter may also be trained in the co-training setup and is thus referred to as π_V hereafter.

Relative Lyapunov Decrease Loss penalizes trajectories along which the Lyapunov candidate fails to decay. For each sample, the decrease violation is defined as

$$e_V = (1-\kappa)V_\theta(x) - V_\theta(x_V^+), \tag{6.11}$$

where $\kappa \in (0,1)$ is a hyperparameter that controls the desired rate of decrease. To balance the optimization scale across different magnitudes of V_θ, we normalize this violation to obtain the relative decrease loss:

$$\mathcal{L}_{\text{dec}} = \left[\frac{e_V}{\max(|V_\theta(x)|, \varepsilon_{\text{rel}})}\right]_+, \tag{6.12}$$

where $\varepsilon_{\text{rel}} > 0$ prevents division by zero. Note that the decrease normalization can be omitted depending on the training configuration.

Relative State Invariance Loss penalizes successor states leaving the Lyapunov training domain $\mathcal{X}_V$. To evaluate the relative one-step state-constraint violation, the scaling matrix $S_{\text{range}} = \text{diag}(\bar{x}_V - \underline{x}_V)$ is utilized. The loss per sample for the next state x_V^+ is computed as

$$\mathcal{L}_{\text{inv}} = \left\| S_{\text{range}}^{-1} [x_V^+ - \bar{x}_V]_+ \right\|_1 + \left\| S_{\text{range}}^{-1} [\underline{x}_V - x_V^+]_+ \right\|_1. \tag{6.13}$$

Combined ρ-Gated Condition Loss is the sample-wise ρ-gated combination of the relative Lyapunov decrease loss from (6.12) and the relative state invariance loss from (6.13),

$$\mathcal{L}_{\text{cond}} = \mathcal{L}_{\text{dec}} + \omega_{\text{inv}} \cdot \mathcal{L}_{\text{inv}}, \tag{6.14}$$

where ω_{inv} is a hyperparameter balancing the relative importance of the two loss components. However, to apply the ρ-gate appropriately, $\mathcal{L}_{\text{cond}}$ is weighted for each sample using the soft sublevel weight

$$w_\rho = \sigma\left(s \cdot \frac{\rho - V_\theta(x)}{\max(\rho, \varepsilon_{\text{rel}})}\right), \tag{6.15}$$

where $s > 0$ denotes the gate sharpness. This smooth formulation ensures that the weights inside $\mathcal{V}_\rho$ are ≈ 1, whereas the weights for states far outside of ρ approach zero. Evaluating these terms on a mini-batch $\mathcal{B}_\rho$ drawn from the combined state pools $\mathcal{S} \cup \mathcal{C}$ (details in Section 6.2.3), the weighted mean of the combined condition loss results in the final loss formulation:

$$\mathcal{L}_{\rho\text{-gated}} = \frac{\sum_{\mathcal{B}_\rho} w_\rho \mathcal{L}_{\text{cond}}}{\sum_{\mathcal{B}_\rho} w_\rho}. \tag{6.16}$$

The complete data flow, from evaluating the batch constraints to computing this final gated loss, is summarized in Fig. 6.3.

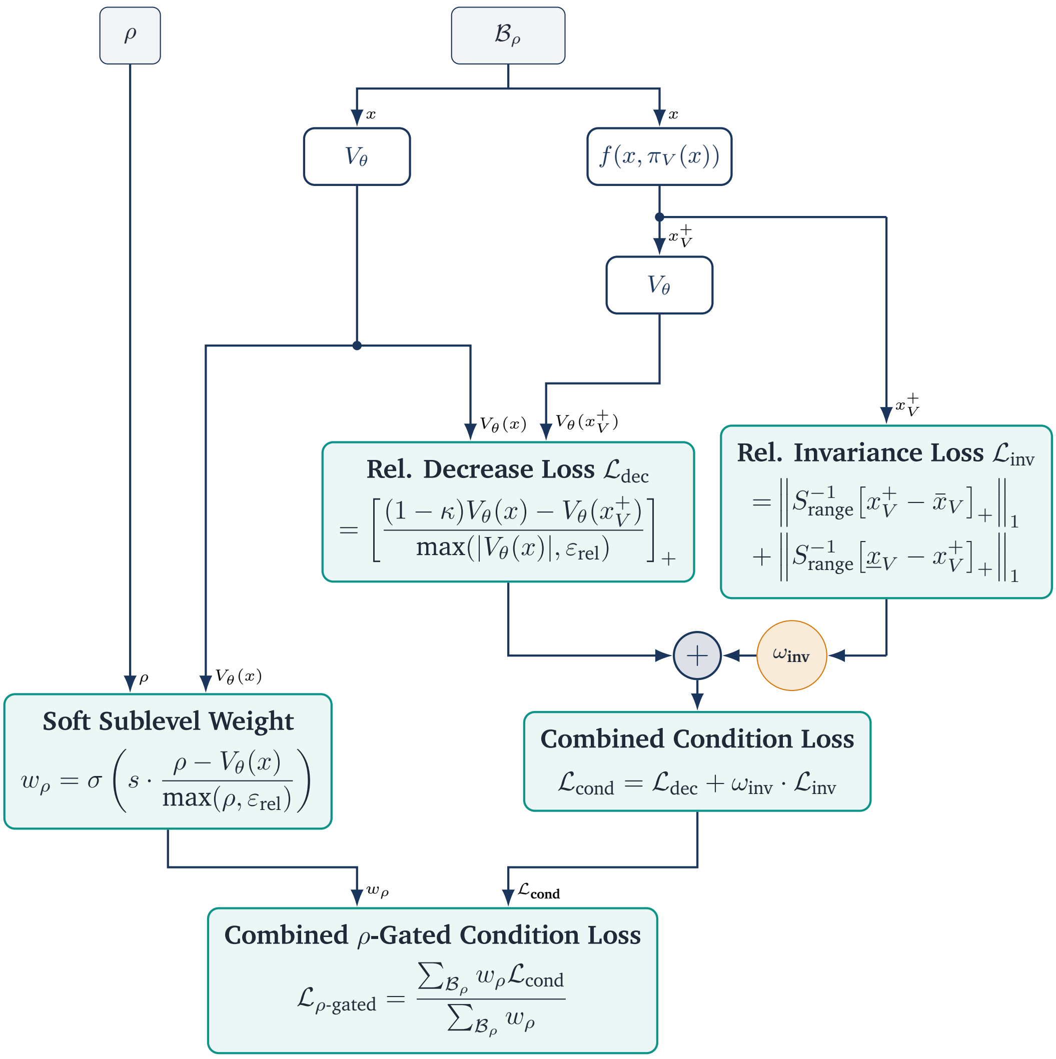

Figure 6.3: Combined gated condition loss $\mathcal{L}_{\rho\text{-gated}}$

ROA Loss tries to enlarge the region of attraction by uniformly drawing a batch $\mathcal{B}_{\text{ROA}}$ from $\mathcal{X}_V \setminus \xi\mathcal{X}_V$, where $\xi \in [0, 1)$ is a scaling factor defining the inner boundary of the shell, as shown in Fig. 6.4. The loss is then evaluated over these candidates using [6]

$$\mathcal{L}_{\text{ROA}} = \frac{1}{|\mathcal{B}_{\text{ROA}}|} \sum_{x \in \mathcal{B}_{\text{ROA}}} \ln \left(\left[\frac{V_\theta(x)}{\rho} - 1 \right]_+ + 1 \right). \tag{6.17}$$

Consequently, the loss function penalizes shell samples that fall outside the current ρ subsets. As the estimated region of attraction expands, fewer samples are penalized, causing the loss to decrease.

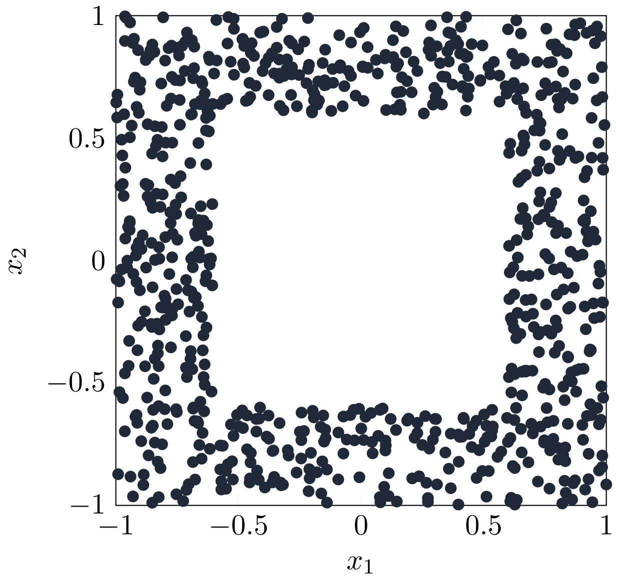

Figure 6.4: Illustration of uniformly drawn samples in the shell batch $\mathcal{B}_{\text{ROA}}$ with $\xi = 0.6$.

LIRPA-Condition Loss makes the Lyapunov function more verifier-friendly by penalizing formal violations of the Lyapunov decrease condition as defined in (6.11). These violations are identified using the LIRPA framework [8]. To reduce the computational cost during training, LIRPA is applied only to a heuristic subset of regions. Specifically, a hyper-rectangular cell is selected if at least one of its 2^{n_x} corners, or optionally its center point x_{center}, lies within the sublevel set $\mathcal{V}_\rho$ (i.e., $\min(V(x)) \leq \rho$ evaluated over these points). For each selected region, LIRPA then provides a formal lower bound $e_{V,\text{lower}}$ on the signed Lyapunov decrease $-e_V$. After evaluation, the LIRPA-condition violation for each region is computed as

$$\mathcal{M}_{\text{L}} = \frac{\ln([-e_{V,\text{lower}}]_+ + 1)}{\max(V_\theta(x_{\text{center}}), \varepsilon_{\text{rel}})}. \tag{6.18}$$

The final LIRPA-condition loss is then computed as the weighted mean over all evaluated regions, where the weight is defined using an exponential function of the squared Euclidean distance between the region center x_{center} and the equilibrium x^*:

$$w_{\text{L}} = \exp\left(-\alpha_{\text{L}} \cdot \|x_{\text{center}} - x^*\|_2^2\right), \tag{6.19}$$

where $\alpha_{\mathrm{L}} > 0$ is a hyperparameter controlling the decay rate of the weight. Finally, the LIRPA-condition loss over N_{L} regions is computed as

$$\mathcal{L}_{\mathrm{L}} = \frac{\sum_{i=1}^{N_{\mathrm{L}}} w_L^{(i)} \cdot \mathcal{M}_{\mathrm{L}}^{(i)}}{\sum_{i=1}^{N_{\mathrm{L}}} w_L^{(i)}}. \tag{6.20}$$

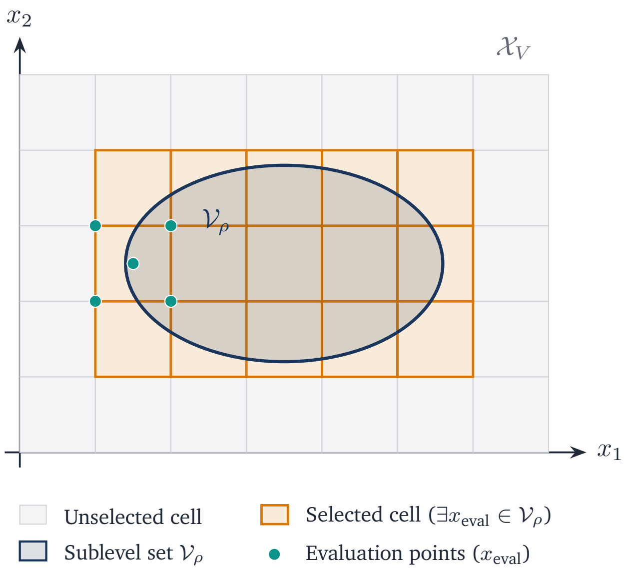

Figure 6.5: Lirpa regions selection with $\mathcal{V}_\rho$.

Lyapunov Scaling Loss prevents numerical collapse to zero by penalizing deviations from the initial average magnitude over a quasi-Monte Carlo batch $\mathcal{B}_V \subset \mathcal{X}_V$, sampled via a low-discrepancy Sobol sequence [9], as illustrated in Fig. 6.6. The logarithm of the mean Lyapunov value over this batch is defined as

$$\ell_V = \ln\left(\frac{1}{|\mathcal{B}_V|} \sum_{x \in \mathcal{B}_V} V_\theta(x)\right), \tag{6.21}$$

leading to the scaling loss formulation

$$\mathcal{L}_V = \left(\ell_{V,0} - \ell_V\right)^2. \tag{6.22}$$

The reference value $\ell_{V,0}$ is determined once using the initial Lyapunov candidate prior to training. To prevent the Lyapunov candidate from overfitting to a static point cloud, a new batch $\mathcal{B}_V$ is resampled from $\mathcal{X}_V$ with a specified probability at each optimization step.

Policy Scaling Loss encourages the learned policy π_V to remain close to the output of the original imitation controller π, which is based on the MPC teacher. A quasi-Monte Carlo batch $\mathcal{B}_{\mathrm{pol}} \subset \mathcal{X}_V$ is sampled using a low-discrepancy Sobol sequence [9]. To prevent overfitting to a static set of states,

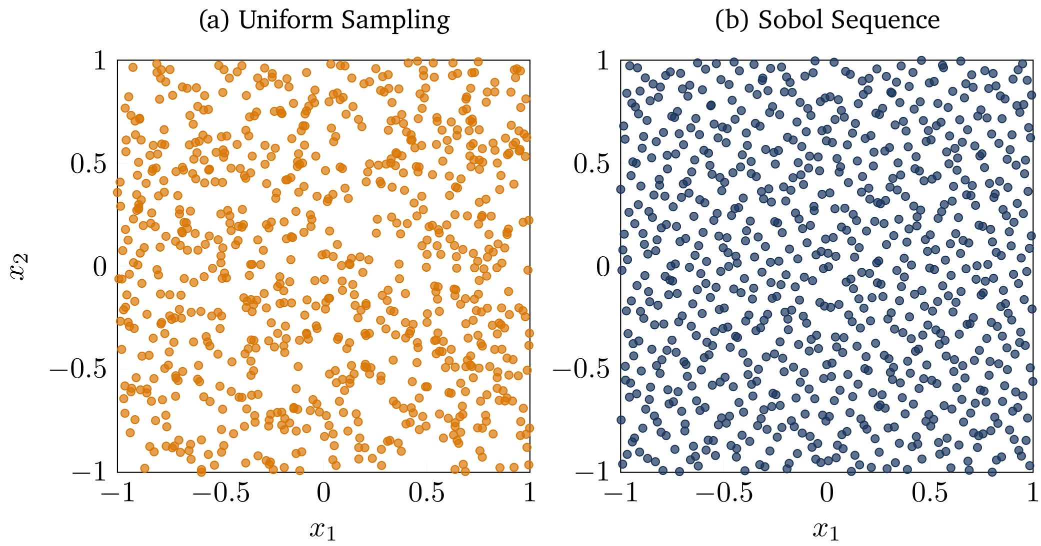

Figure 6.6: Comparisson of Sobol and uniform sampling in two dimensions.

this batch also is resampled from $\mathcal{X}_V$ with a given probability each optimization step. The normalized deviation between the learned policy and the imitation controller over this batch is formulated as

$$\mathcal{L}_{\text{pol}} = \frac{\sum_{x \in \mathcal{B}_{\text{pol}}} \|\pi_V(x) - \pi(x)\|_2^2}{\sum_{x \in \mathcal{B}_{\text{pol}}} \|\pi(x)\|_2^2} \tag{6.23}$$

R-Matrix Loss regularizes the learnable matrix R_θ by penalizing deviations of its Frobenius norm from its initial value $\ell_{R,0}$:

$$\mathcal{L}_R = (\ell_{R,0} - \|R\|_F)^2 \,. \tag{6.24}$$

However, $\ell_{R,0} = \|R_\theta\|_F$ is automatically computed, based on the initial $R_{\theta,0}$ matrix.

Overall Lyapunov Loss represents the combined objective function, summing all previously defined weighted components, given by

$$\mathcal{L}_{\text{total}} = \sum_m \omega_m \mathcal{L}_m \tag{6.25}$$

where the weights $\omega_m > 0$, with $m \in \{\rho\text{-gated, ROA, L}, V, \text{pol}, R\}$ are hyperparameters that balance the relative importance of the individual loss components.

6.2.3 Falsification-Guided Training

Relying solely on uniform state sampling is not sufficient to reliably detect violations of the Lyapunov condition, especially in higher-dimensional state spaces **counterexampleRef**. To overcome this problem, falsification-guided training actively searches for adversarial counterexamples and incorporates them into the training distribution. This is made possible by an adversarial replay buffer that reconciles the exploration of the estimated attractor region with the targeted exploitation of known error modes.

PGD Falsification is utilized to identify counterexamples that violate the Lyapunov conditions and iteratively inject them into the counterexample pool $\mathcal{C}$. Specifically, an L_∞-Projected Gradient Descent (PGD) is employed to find adversarial states $x_{\text{adv}} \in \mathcal{X}_V$ that maximize the condition violation defined in (6.14) and (6.15). The per-sample PGD objective is defined as

$$J_{\text{PGD}} = -w_\rho \mathcal{L}_{\text{cond}}. \tag{6.26}$$

The adversarial search is initialized using the explorative pool $\mathcal{S}$, with up to 25% of the batch supplemented by existing counterexamples to encourage local refinement. The algorithm minimizes J_{PGD} by iteratively updating the input states along the sign of their local gradient with a constant step size α, followed by a projection back onto the training bounds $\mathcal{X}_V$. To retain the strongest violations and mitigate the effects of overshooting, the state yielding the minimum objective value across all K iterations is preserved for each candidate. Ultimately, only states strictly within $\mathcal{V}_\rho$ that exceed the violation tolerance and lie outside the origin exclusion region are retained as counterexamples.

Adversarial Replay Buffer manages the sample distribution for the ρ-gated condition loss $\mathcal{L}_{\rho\text{-gated}}$. It maintains two distinct state pools: an explorative pool $\mathcal{S}$ and a counterexample pool $\mathcal{C}$.

As outlined in Algorithm 3, the explorative state pool $\mathcal{S}$ is repopulated at each outer epoch to balance global exploration with local refinement. First, candidate states $\mathcal{B}_\mathcal{S}$ are oversampled uniformly from the training domain $\mathcal{X}_V$. From these candidates, $N_\mathcal{S}$ states are probabilistically sampled based on a sigmoid weighting function centered around an expanded sublevel boundary $m \cdot \rho$ (with margin $m > 1$). This weighting smoothly penalizes states further outside the estimated region of attraction, significantly reducing their likelihood of being selected.

Algorithm 3: Probabilistic Explorative State Pool Resampling

Input: Lyapunov function V_θ, current ROA estimate ρ, margin m, sharpness s, target size $N_\mathcal{S}$
Output: Updated state pool $\mathcal{S}$ of size $N_\mathcal{S}$

1 Generate $4N_\mathcal{S}$ candidates: $\mathcal{B}_\mathcal{S} \sim \mathcal{U}(\mathcal{X}_V)$;

2 For all $x \in \mathcal{B}_\mathcal{S}$, assign weight $w(x) \leftarrow \sigma\left(s\,\frac{m\cdot\rho - V_\theta(x)}{\max(\rho, 10^{-9})}\right) + \varepsilon$;

3 $\mathcal{S} \leftarrow$ Sample $N_\mathcal{S}$ states from $\mathcal{B}_\mathcal{S}$ with probability $P(x) \propto w(x)$;

The counterexample pool $\mathcal{C}$ stores the critical violations identified during the PGD falsification step. A age-tracking mechanism limits sample lifespan, preventing overfitting outdated violations. Each injected counterexample is assigned an initial age $\tau = 0$, which is incremented after every subsequent falsification phase. Once a counterexample reaches the maximum age $\tau_{\max}$, it is removed from the pool. Consequently, the condition loss continuously penalizes recent violations while discarding resolved edge cases as the Lyapunov candidate evolves.

6.2.4 ρ-Sublevel Estimation

explain why exactly we need it

To compute the loss, particularly the ρ-gated condition loss in (6.16), the target level ρ must be continuously estimated during training. Since the goal is to find the largest level set of the Lyapunov candidate $V_\theta(x)$ that lies entirely within the defined training bounds $\mathcal{X}_V$, a level set can only escape the safe region by intersecting its boundary $\partial\mathcal{X}_V$. Therefore, the maximum certifiable ρ is mathematically strictly determined by the minimum value of the Lyapunov candidate at its boundary. According to the formulation proposed by [6], it is sufficient to evaluate $V_\theta(x)$ only on the boundary $\partial\mathcal{X}_V$ rather than sampling the entire state space. However, since the boundary is a high-dimensional surface and the neural network parameterized in $V_\theta(x)$ forms a highly non-convex landscape, simple uniform sampling on the boundary is insufficient to find the true minimum. Therefore, a Projected Gradient Descent (PGD) approach is employed to actively search for the worst-case boundary points. Starting from uniformly sampled points $\mathcal{B}_\partial \sim \mathcal{U}(\partial\mathcal{X}_V)$, PGD iteratively minimizes $V_\theta(x)$ by taking steps along the negative gradient and projecting the updated states strictly back onto the boundary surfaces. To actively encourage the region of attraction to expand over the course of the co-training process, the formally certifiable limit is intentionally overestimated, as suggested in [6]. Specifically, a target ρ is computed by applying a growth factor $\gamma > 1$ to the boundary minimum found via PGD:

$$\tilde{\rho} = \max\left(\rho_{\min}, \gamma \cdot \min_{x \in \mathcal{B}_\partial} V_\theta(x)\right). \tag{6.27}$$

By targeting a value for $\tilde{\rho}$ that is slightly above the currently safe maximum, the formulation deliberately leads to small violations near the boundary. These violations are then captured by the aforementioned adversarial replay buffer, effectively forcing the optimization to continuously expand the sublevel set outward. Finally, to ensure numerical stability and prevent irregular jumps in the loss landscape caused by fluctuating local minima across different batches, the ρ value actually used for gating is updated using an exponential moving average (EMA):

$$\rho = \beta\rho + (1-\beta)\tilde{\rho}, \tag{6.28}$$

where $\beta \in [0, 1)$ is the decay rate.

6.2.5 Overall Training Procedure

The complete falsification-guided training procedure is illustrated in Fig. 6.7. It consists of an outer loop for data generation and an inner loop for network optimization.

At the beginning of each outer epoch, boundary samples $\mathcal{B}_\partial$ are used to estimate the current sublevel bound ρ, and the ROA shell batch $\mathcal{B}_{\text{ROA}}$ is redrawn. Based on the updated estimation of ρ, the active LiRPA regions are selected and PGD-based falsification is performed within the corresponding sublevel set. The resulting counterexamples are added to $\mathcal{C}$, after which the explorative state pool $\mathcal{S}$ is resampled.

The inner loop is executed K_{steps} times per outer epoch. At each step, a mini-batch $\mathcal{B}_\rho$ is sampled from $\mathcal{S} \cup \mathcal{C}$ and evaluated together with the active LiRPA regions, the ROA shell batch, and Sobol batches $\mathcal{B}_V$ and $\mathcal{B}_{\text{pol}}$. Their corresponding loss terms are combined into $\mathcal{L}_{\text{total}}$, which is minimized by backpropagation with respect to the trainable network parameters. Repeating this procedure over N_{outer} epochs progressively reduces Lyapunov-condition violations and enlarges the estimated certified region of attraction.

arrow from boundary batch to rho

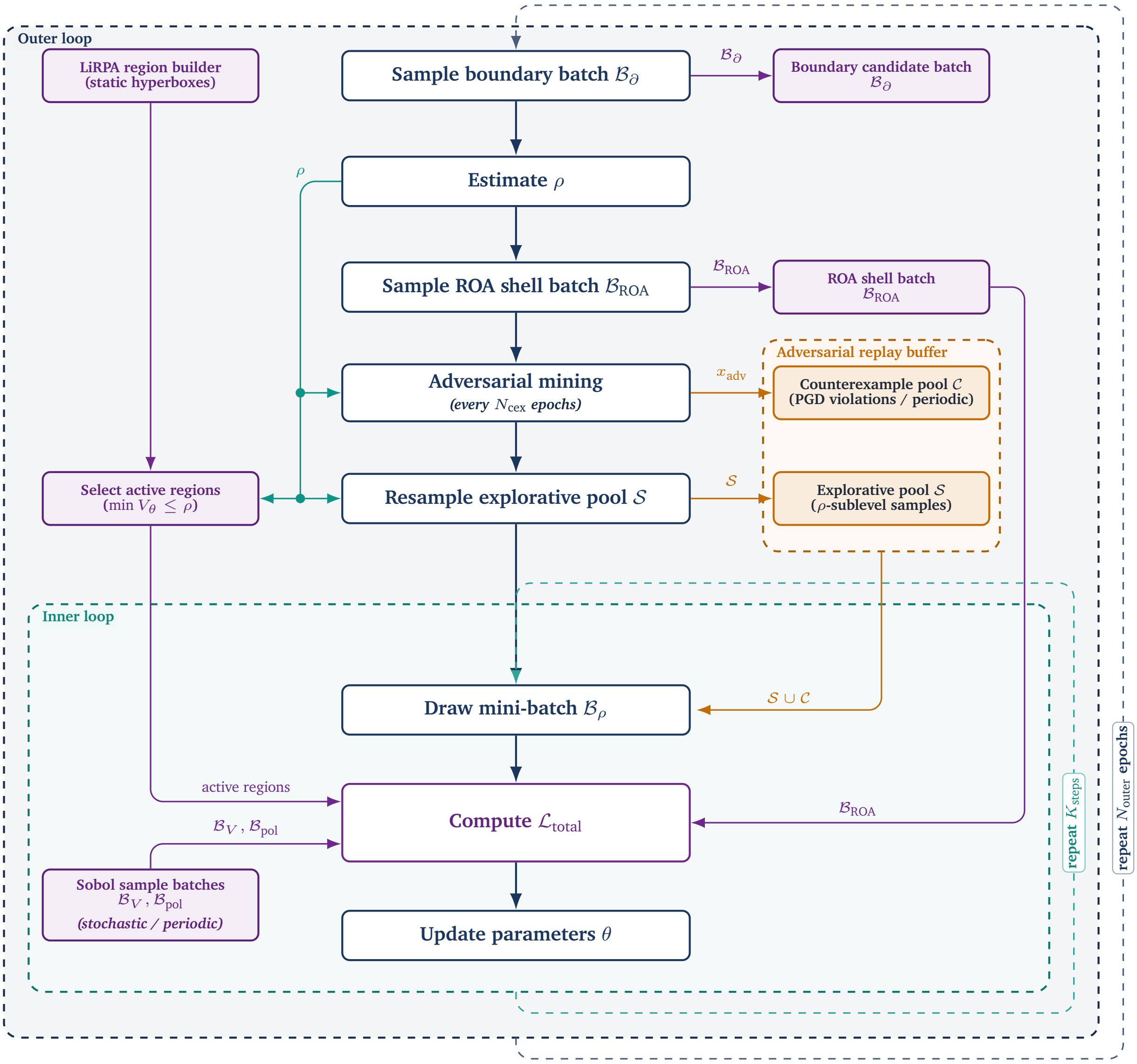

Figure 6.7: Overview of the falsification-guided training framework. The architecture is divided into an outer epoch-level loop responsible for state space exploration, boundary estimation, and adversarial mining, and an inner step-level loop that optimizes the network parameters based on the generated sample pools.

6.3 Formal Verification Methodology

This section adapts the search algorithm for the maximal certifiable sublevel set $\mathcal{V}_\rho$ proposed in [6]. Specifically, for a given target level $\rho > 0$, formal verification aims to certify that all states within the candidate region $\mathcal{V}_\rho := \{x \in \mathcal{B}_C \mid V_\theta(x) \leq \rho\}$ satisfy controlled regional invariance and exponential decay:

$$V_\theta(x) \leq \rho \implies \begin{cases} x^+ \in \mathcal{B}_C, \\ V_\theta(x^+) \leq (1-\kappa)V_\theta(x), \end{cases} \tag{6.29}$$

where $\kappa \in (0,1)$ specifies the targeted convergence rate and $x^+ = f(x, \pi_\theta(x))$ denotes the discrete one-step state transition.

Before evaluating the stability conditions, the equilibrium x^* must be explicitly excluded from the bounded certification domain $\mathcal{B}_C := \{x \in \mathbb{R}^n \mid \underline{x}_C \leq x \leq \bar{x}_C\} \subseteq \mathcal{B}_V$. By construction, $V_\theta(x^*) = 0$ and $x^+ = x^*$ hold at the equilibrium, causing the strict positivity and strict decrease conditions to evaluate to false at this exact point. Since these conditions are only required to hold for $x \neq x^*$, and positive definiteness is structurally enforced by the network architecture, formal verification is instead performed over the punctured evaluation domain $\mathcal{B}_{\text{eval}} := \mathcal{B}_C \setminus \mathcal{B}_{\text{hole}}$, where $\mathcal{B}_{\text{hole}}$ denotes a small hyper-rectangular region centered at x^*. For the solver to handle the punctured certification domain, $\mathcal{B}_{\text{eval}}$ is split into multiple subregions, as shown in Fig. 6.8.

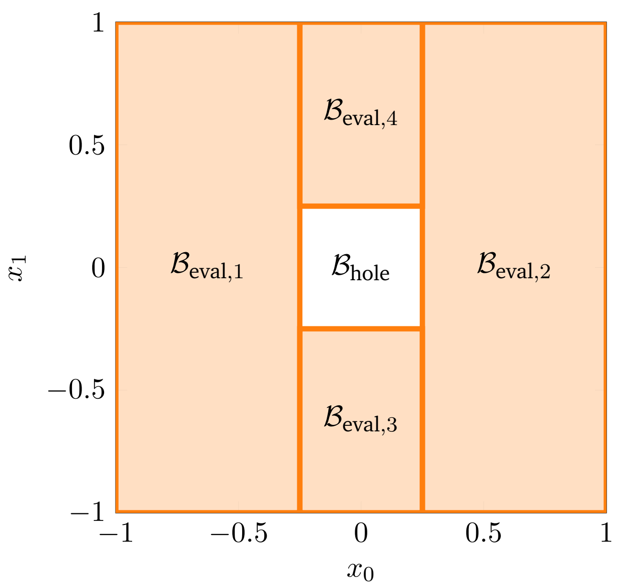

Figure 6.8: 2D Hole exclusion example with $\mathcal{B}_{\text{hole}} := \{x \in \mathbb{R}^2 \mid \|x\|_\infty < 0.25\}$.

To certify property (6.29) over $\mathcal{B}_{\text{eval}}$ using $\alpha\beta$-CROWN [8], [10], the implication is rewritten into a disjunctive formulation supported by the verifier: a state must either lie outside $\mathcal{V}_\rho$ or satisfy all stability conditions inside $\mathcal{B}_{\text{eval}}$. The individual logical terms are defined as follows:

$$\Phi_{\text{sub}} \equiv V_\theta(x) > \rho, \tag{6.30a}$$
$$\Phi_{\text{pos}} \equiv V_\theta(x) > 0, \tag{6.30b}$$
$$\Phi_{\text{inv}} \equiv \underline{x}_C \leq x^+ \leq \bar{x}_C, \tag{6.30c}$$
$$\Phi_{\text{dec}} \equiv (1-\kappa)V_\theta(x) > V_\theta(x^+). \tag{6.30d}$$

Here, (6.30a) excludes states outside the target sublevel set, while (6.30b) to (6.30d) explicitly enforce positive definiteness, state constraint invariance, and exponential decrease. Combining these terms yields the overall state-wise specification:

$$\Phi(x;\rho) \equiv \Phi_{\text{sub}} \vee \left(\Phi_{\text{pos}} \wedge \Phi_{\text{inv}} \wedge \Phi_{\text{dec}}\right). \tag{6.31}$$

To determine whether a candidate level ρ is valid across the entire continuous evaluation domain, the global verification condition $\Psi(\rho)$ evaluates whether $\Phi(x;\rho)$ holds for all states:

$$\Psi(\rho) \equiv \forall x \in \mathcal{B}_{\text{eval}} : \Phi(x;\rho). \tag{6.32}$$

explain algorithm

Algorithm 4: Certified Value Search via Scaling and Bisection

Input: $\rho_0, \rho_{\text{min}}$, evaluator Ψ, scale $s > 1$, tol ε, limits $N_{\text{scale}}, N_{\text{bisect}}$
Output: Max certified value ρ^*

$\rho \leftarrow \max(\rho_0, \rho_{\text{min}})$;
`// Phase 1: Scaling`
if $\Psi(\rho)$ **then**
  for $k \leftarrow 1$ **to** N_{scale} **do**
    if $\neg\Psi(\rho)$ **then**
      break;
    end
    $\rho_{\text{lo}} \leftarrow \rho$; $\rho \leftarrow \rho \cdot s$;
  end
  $\rho_{\text{up}} \leftarrow \rho$;
else
  for $k \leftarrow 1$ **to** N_{scale} **do**
    if $\Psi(\rho) \vee \rho = \rho_{\text{min}}$ **then**
      break;
    end
    $\rho_{\text{up}} \leftarrow \rho$; $\rho \leftarrow \max(\rho_{\text{min}}, \rho/s)$;
  end
  if $\neg\Psi(\rho)$ **then**
    return 0 ;   `// No valid value` $\geq \rho_{\text{min}}$
  end
  $\rho_{\text{lo}} \leftarrow \rho$;
end
`// Phase 2: Bisection`
for $k \leftarrow 1$ **to** N_{bisect} **do**
  if $\rho_{up} - \rho_{lo} \leq \varepsilon$ **then**
    break;
  end
  $\rho_{\text{mid}} \leftarrow \frac{1}{2}(\rho_{\text{lo}} + \rho_{\text{up}})$;
  if $\Psi(\rho_{mid})$ **then**
    $\rho_{\text{lo}} \leftarrow \rho_{\text{mid}}$;
  else
    $\rho_{\text{up}} \leftarrow \rho_{\text{mid}}$;
  end
end
return ρ_{lo};

7 Experimental Evaluation and Certification Results

7.1 System Modeling and Problem Formulation

For our experimental setup, two benchmark systems are selected: the double integrator and the cartpole. These systems provide a good balance between simplicity and complexity allowing us to certify the stability of the closed-loop system.

7.1.1 Double Integrator

The double integrator is a simple second-order linear system; therefore, it serves as the ideal baseline system in the learning and certification pipeline. Discretizing the continuous-time dynamics using the explicit Euler method with a sampling time of $\Delta t = 0.1$, leads to the discrete-time state-space equation:

$$x_{k+1} = f_{\mathrm{DI},d}(x_k, u_k) = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} 0 \\ \Delta t \end{bmatrix} u_k \tag{7.1}$$

where $x_k = \begin{bmatrix} p_k & v_k \end{bmatrix}^\top$ is the state vector containing the position and velocity of the system, respectively, and u_k denotes acceleration.

The *MPC setup* is formulated according to the problem in (3.4) subject to the double-integrator dynamics in (7.1). A prediction horizon of $N = 20$ steps is chosen, and the states and inputs are penalized using the weighting matrices $Q = \mathrm{diag}(15, 1)$ and $R = 0.1$.

7.1.2 Cartpole

The inverted pendulum on a cart, commonly referred to as the cartpole system, is used as a nonlinear benchmark system in this work. There are four states in the system: the cart position p, the cart velocity v, the pendulum angle θ, and the pendulum angular velocity ω. Here, the state vector is defined as $x = \begin{bmatrix} p & v & \theta & \omega \end{bmatrix}^\top$, and the input u is the force F applied to the cart. This leads to the following continuous-time dynamics of the cartpole system:

$$f_{\text{CP}}(x,u) = \begin{bmatrix} \dot{p} \\ \dot{v} \\ \dot{\theta} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} v \\ \frac{F + ml\omega^2 \sin(\theta) - mg\sin(\theta)\cos(\theta)}{M + m - m\cos^2(\theta)} \\ \omega \\ \frac{-F\cos(\theta) - ml\omega^2 \sin(\theta)\cos(\theta) + (M+m)g\sin(\theta)}{l(M + m - m\cos^2(\theta))} \end{bmatrix} \tag{7.2}$$

where $M = 1\,\text{kg}$ is the mass of the cart, $m = 0.1\,\text{kg}$ is the mass of the pendulum, $l = 0.8\,\text{m}$ is the length of the pendulum, and $g = 9.81\,\text{m/s}^2$ is the acceleration due to gravity. Discretizing the continuous-time dynamics using the explicit Euler method with a sampling time of $\Delta t = 0.05\,\text{s}$, leads to the discrete-time state-space equation:

$$x_{k+1} = f_{\text{CP},d}(x_k, u_k) = x_k + \Delta t f_{\text{CP}}(x_k, u_k) \tag{7.3}$$

For the nonlinear benchmark, a prediction horizon of $N = 40$ stages is chosen. The system states and control inputs are penalized within the optimization framework of (3.4) using the weighting matrices $Q = \text{diag}(10, 1, 10, 0.01)$ and $R = 0.5$, respectively, subject to the dynamics defined in (7.3).

7.2 Linear System Results: Double Integrator

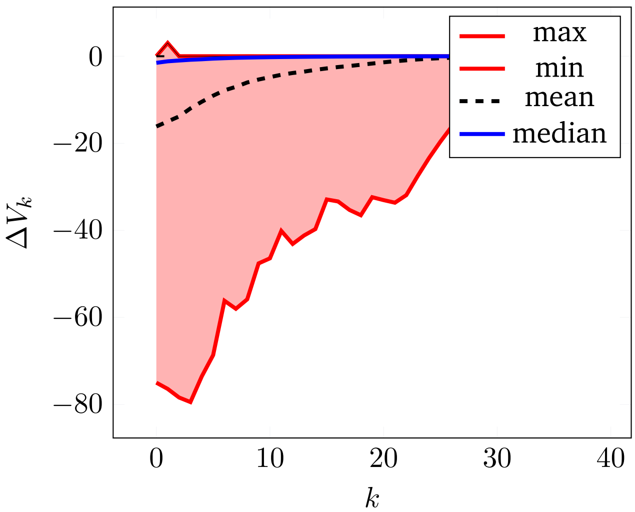

Figure 7.1: Double integrator cost descent. $\Delta V = V_N(x_{k+1}) - V_N(x_k)$

7.3 Nonlinear System Results: Cartpole

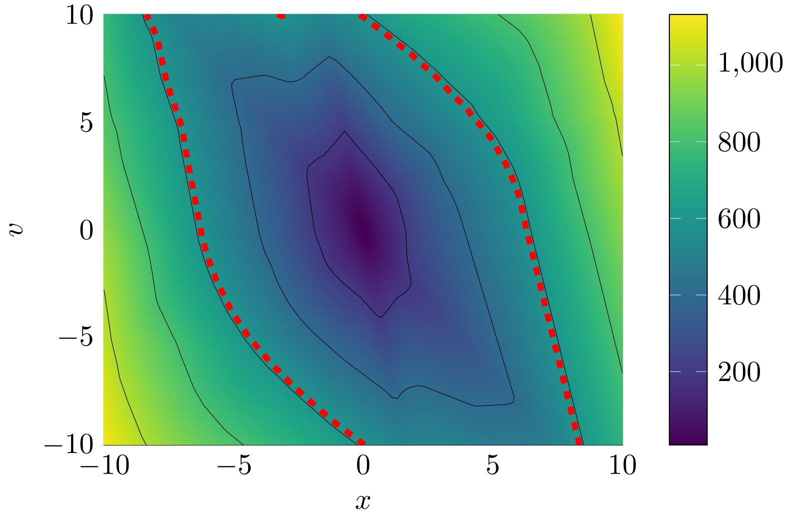

Figure 7.2: Double integrator Lyapunov landscape and ROA: value

8 Discussion

8.1 Qualitative Discussion of Results

8.2 Challenges with Nonlinear Dynamics

8.3 Scalability and Limitations

9 Conclusion

A Appendix

A.0.1 Imitation Learning

Note: If the MPC problem becomes infeasible at any time during one trajectory, all training pairs associated with that trajectory are discarded.

To reduce the overrepresentations of regions with high sampling rates, near-duplicate state-action pairs are removed during preprocessing. The concatenated state-action vectors are normalized and mapped onto an adaptive voxel grid. For each occupied voxel, at most one representative pair is retained.

Bibliography

[1] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Chichester, UK: Wiley, 2016, 1 p., ISBN: 978-1-119-12150-3 978-1-119-12151-0 978-1-119-12153-4. DOI: 10.1002/9781119121534.

[2] J. B. Rawlings, D. Q. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation and Design*, 2nd edition. Santa Barbara: Nob Hill Publishing, LLC, 2020, 623 pp., ISBN: 978-0-9759377-5-4.

[3] L. Grüne and J. Pannek, *Nonlinear Model Predictive Control: Theory and Algorithms* (Communications and Control Engineering), 2nd ed. 2017. Cham: Springer, 2017, 456 pp., ISBN: 978-3-319-46024-6. DOI: 10.1007/978-3-319-46024-6.

[4] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*, Second Edition. Cham: Springer International Publishing AG, 2023, 529 pp., ISBN: 978-3-031-29642-0 978-3-031-29641-3.

[5] S. Kollmannsberger, D. D´Angella, M. Jokeit, and L. Herrmann, *Deep Learning in Computational Mechanics: An Introductory Course* (Studies in Computational Intelligence volume 977). Cham: Springer, 2021, 104 pp., ISBN: 978-3-030-76587-3 978-3-030-76589-7.

[6] L. Yang, H. Dai, Z. Shi, C.-J. Hsieh, R. Tedrake, and H. Zhang. "Lyapunov-stable Neural Control for State and Output Feedback: A Novel Formulation". arXiv: 2404.07956 [cs.LG]. (Jun. 5, 2024), [Online]. Available: http://arxiv.org/abs/2404.07956 (visited on 06/24/2026), pre-published.

[7] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization", presented at the International Conference on Learning Representations, 2015. arXiv: 1412.6980 [cs.LG]. [Online]. Available: http://arxiv.org/abs/1412.6980 (visited on 08/06/2026).

[8] K. Xu, H. Zhang, S. Wang, *et al.*, "Fast and Complete: Enabling Complete Neural Network Verification with Rapid and Massively Parallel Incomplete Verifiers", in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=nVZtXBI6LNn (visited on 06/24/2026).

[9] S. Joe and F. Y. Kuo, "Constructing Sobol Sequences with Better Two-Dimensional Projections", *SIAM Journal on Scientific Computing*, vol. 30, no. 5, pp. 2635–2654, Jan. 2008, ISSN: 1064-8275, 1095-7197. DOI: 10.1137/070709359. [Online]. Available: http://epubs.siam.org/doi/10.1137/070709359 (visited on 08/03/2026).

[10] S. Wang, H. Zhang, K. Xu, *et al.*, "Beta-CROWN Efficient bound propagation with per-neuron split constraints for complete and incomplete neural network verification", *Advances in Neural Information Processing Systems*, vol. 34, 2021.